

Reviewer Pack — Kruskal-Katona Shadow Defect Lower-Bounds Monotone-KW Depth

Entry 37788821c84c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 15:27:44 UTC

Kruskal-Katona Shadow Defect Lower-Bounds Monotone-KW Depth

Entry ID: 37788821c84c **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 15:27:44 UTC

1. Conjecture

Field A (mathematical branch): Extremal set theory: the Kruskal-Katona shadow theorem and cascade representation of k -uniform antichains in the Boolean lattice (Kruskal 1963; Katona 1968; Frankl tradition). The ‘KK shadow defect’ $\Delta_{\text{KK}}(A, k) := |\text{partial}(A)| - \text{KK_lower}(|A|, k)$ — slack between the actual $(k-1)$ -shadow of a k -family A and its cascade-extremal lower bound — is a classical extremal-combinatorics invariant, computable in $O(|A| \cdot n)$ time, that has rarely been deployed against KW games (distinct from Bollobás set-pair inequalities used in monotone-circuit Razborov-approximation arguments). **Field B** (complexity object): Monotone Karchmer-Wigderson depth $D_m(f)$ of monotone Boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$, equivalently monotone formula depth in the $\{\text{AND}, \text{OR}\}$ -basis via the Karchmer-Wigderson equivalence; the KW relation is the bipartite search problem on (minterm, maxterm) pairs of f .

Statement:

Let $f: \{0,1\}^n \rightarrow \{0,1\}$ be monotone with minterm family $M(f)$ sliced by Hamming weight as $M(f) = \bigsqcup_k M_k(f)$, and let $\text{partial}(M_k)$ denote its $(k-1)$ -shadow in 2^1 . Define the slice-wise shadow defect $\Delta_{\text{KK}}(f) := \max_{1 \leq k \leq n} (|\text{partial}(M_k(f))| - \text{KK_lower}(|M_k(f)|, k))$, where $\text{KK_lower}(m, k)$ is the Kruskal-Katona cascade bound on the shadow of any k -uniform antichain of size m . Then for every monotone f on $n \leq 20$ with $|M(f)| \geq 2$, $D_m(f) \geq \lceil \log_2(\Delta_{\text{KK}}(f) + 2) \rceil$.

Rationale (proposer’s reasoning):

A monotone formula of depth d induces a balanced KW protocol whose 2^d leaves each fix a single coordinate witness for an entire surviving minterm-by-maxterm rectangle; this forces every weight-slice of $M(f)$ to admit a covering by at most 2^d coordinate-stars, which in turn forces the slice to be near-compressed (low shadow defect) by a Frankl-style compression argument. KK-extremal slices correspond exactly to threshold-like minterm families which are known to admit $O(\log^2 n)$ monotone depth, while large defect at any slice is a structural irregularity invisible to the natural-proofs largeness condition (the property targets monotone depth only and is rare on random monotone clutters). The bound uses no relativization or algebrization, and stays inside the KW framework’s safe directions.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

¹_n

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1aeae8b04e64e64b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 168 monotone f on $n=4$, all 7581 on $n=5$, and 2000 random monotone clutters each for $n=6$ and $n=7$ (total ~ 11749 instances), the bound $D_m(f) \geq \text{ceil}(\log_2(\Delta_{KK}(f)+2))$ must hold for every single instance. Report Spearman ρ and tightness histogram.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.88	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Kruskal-Katona shadow monotone formula depth Karchmer-Wigderson - cascade representation shadow lower bound monotone circuit complexity - k -uniform antichain shadow defect monotone depth minterm

Top relevant hits considered: - [<http://arxiv.org/abs/1809.03967v2>] More Cases Where the Kruskal-Katona Bound is Tight - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/2002.07444v1>] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/1606.05050v1>] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [<http://arxiv.org/abs/2212.11963v3>] Operator relaxation and the optimal depth of classical shadows - [<http://arxiv.org/abs/2501.09994v1>] Multi-Modal Attention Networks for Enhanced Segmentation and Depth Estimation of Subsurface Defects in Pulse Thermograph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import json
import random

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
```

```

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def binomial(n, k):
    if k > n:
        return 0
    num = 1
    denom = 1
    for i in range(k):
        num *= (n - i)
        denom *= (i + 1)
    return num // denom

def power_of_two(x):
    return x and not (x & (x - 1))

def compute_M_k(f, k):
    M_k = set()
    for minterm in f:
        if sum(minterm) == k:
            M_k.add(tuple(sorted(minterm)))
    return M_k

def compute_partial(M_k):
    partial = set()
    for minterm in M_k:
        for i in range(len(minterm)):
            new_minterm = list(minterm)
            new_minterm[i] = 0
            partial.add(tuple(sorted(new_minterm)))
    return partial

def compute_KK_lower(m, k):
    if k == 1:
        return m
    if k == m:
        return 1
    return sum(binomial(m - i, k - 1) for i in range(k))

def compute_D_m(f, M_k):
    n = len(next(iter(M_k)))
    R = set()
    T = set(range(n))
    memo = {}

    def D_m_helper(R, T):
        if (R, T) in memo:
            return memo[(R, T)]
        if not R:
            return 0
        if any(x[i] == 1 and y[i] == 0 for x, y in R):
            return 0
        min_val = float('inf')
        for i in range(n):
            R_0 = {(x, y) for x, y in R if x[i] == 0}

```

```

        R_1 = {(x, y) for x, y in R if x[i] == 1}
        min_val = min(min_val, 1 + max(D_m_helper(R_0, T), D_m_helper(R_1, T)))
    memo[(R, T)] = min_val
    return min_val

return D_m_helper(R, T)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 5]
    if seed == 0:
        n_values.extend([6, 7])
    results = []

    for n in n_values:
        if n == 4 or n == 5:
            # Enumerate all monotone Boolean functions on n
            from itertools import combinations
            M = list(combinations(range(n), n))
            f = [tuple(sorted(m)) for m in M]
        else:
            # Sample random monotone clutters on n
            f = []
            for _ in range(2000):
                k = random.randint(1, n)
                antichain = set(combinations(range(n), k))
                upper_closure = set()
                for x in antichain:
                    for y in M:
                        if all(xi <= yi for xi, yi in zip(x, y)):
                            upper_closure.add(tuple(sorted(y)))
                f.append(upper_closure)

            for f_instance in f:
                M_k = {compute_M_k(f_instance, k) for k in range(n + 1)}
                partial = compute_partial(M_k)
                Delta_KK = max(len(partial) - compute_KK_lower(len(minterms), k) for k, minterms in enumerate(minterms))
                D_m = compute_D_m(f_instance, M_k)

                results.append({
                    "n": n,
                    "Delta_KK": Delta_KK,
                    "D_m": D_m
                })

    if not results:
        return {
            "metric_name": "D_m vs Delta_KK",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    metric_values = [result["Delta_KK"] for result in results]
    D_m_values = [result["D_m"] for result in results]

```

```

from scipy.stats import spearmanr
rho, _ = spearmanr(metric_values, D_m_values)

tightness_ratio = [max(0, (D_m - 1) / (log2(Delta_KK + 2))) for Delta_KK, D_m in zip(metric_values, D_m_values)]
tightness_histogram = {i: tightness_ratio.count(i) for i in set(tightness_ratio)}

return {
    "metric_name": "D_m vs Delta_KK",
    "metric_value": rho,
    "instances_tested": len(results),
    "conjecture_holds": all(D_m >= ceil(log2(Delta_KK + 2)) for Delta_KK, D_m in zip(metric_values, D_m_values)),
    "counterexample": "" if all(D_m >= ceil(log2(Delta_KK + 2)) for Delta_KK, D_m in zip(metric_values, D_m_values)) else None
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
        mean_rho = sum(r["metric_value"] for r in results) / len(results)
        std_rho = (sum((r["metric_value"] - mean_rho) ** 2 for r in results) / len(results)) ** 0.5
        support_fraction = len([r for r in results if "conjecture_holds" in r and r["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction}")
    elif any("counterexample" in r and r["counterexample"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if "counterexample" in r and r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a486861f.py", line 155, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a486861f.py", line 112, in run_trial
    M_k = {compute_M_k(f_instance, k) for k in range(n + 1)}
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a486861f.py", line 42, in compute_M_k
    if sum(minterm) == k:
       ~~~~~
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `compute_M_k` before producing any data, so neither the support condition nor the falsification condition could be evaluated. | next: Fix the minterm representation bug in `compute_M_k` (iterate over bits of the integer-encoded minterm or pass tuple/bitmask consistently) and rerun the full enumeration on $n=4,5$ plus the $n=6,7$ sampled clutters.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	159355
2	preregistration	claude_max	opus	0	0	7696
3	novelty	claude_max	opus	0	0	3152
4	novelty	claude_max	opus	0	0	7194
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16995
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16740
7	judge	claude_max	opus	0	0	5993

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 217125 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/37788821c84c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/37788821c84c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/37788821c84c.tar.gz` (if generated)